

Command Injection Testing:

Command Injection: WAF Coverage and Evasion Testing

I tested DVWA's Command Injection module against the ModSecurity WAF with standard payloads and known evasion techniques, and identified a gap in Wazuh's rule level classification of this attack class.

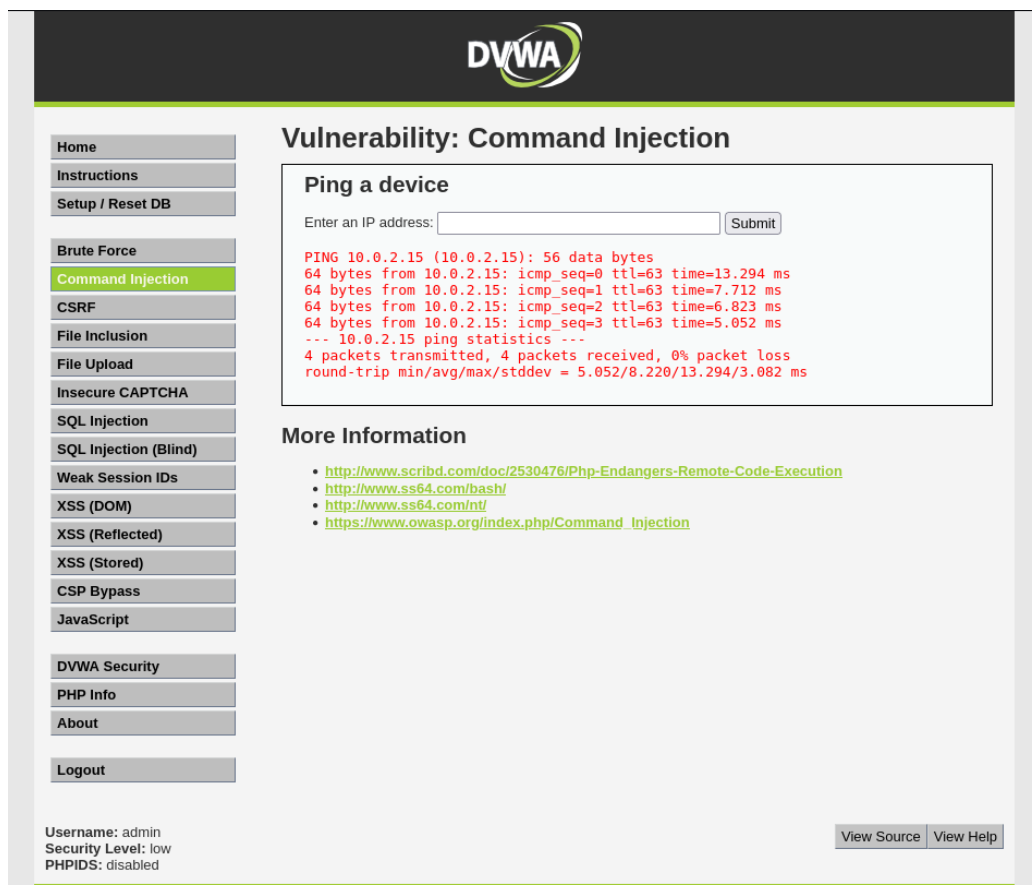
Overview

With SQL injection (Part 2/3) already tested against the lab's ModSecurity WAF and Wazuh SIEM, I extended coverage in this phase to a more severe attack class: OS command injection. Unlike SQL injection, which is scoped to database queries, command injection grants arbitrary code execution on the underlying host at the privilege level of the web server process, a materially higher impact vulnerability class, and a good test of whether the WAF's protection generalizes beyond the attack type it was originally validated against.

I tested five payloads, spanning both standard injection syntax and two common signature based WAF evasion techniques, followed by direct system level verification of what a successful injection would have exposed.

Target and Method

DVWA's Command Injection module (Security Level: Low) accepts an IP address and executes it inside a server side ping command without input sanitization. I routed all requests through the ModSecurity WAF (port 8081) rather than directly to DVWA, consistent with my lab's standard defense in depth testing pattern.



DVWA's Command Injection module, showing the intended behavior: a legitimate IP address returns normal ping output with no WAF interference.

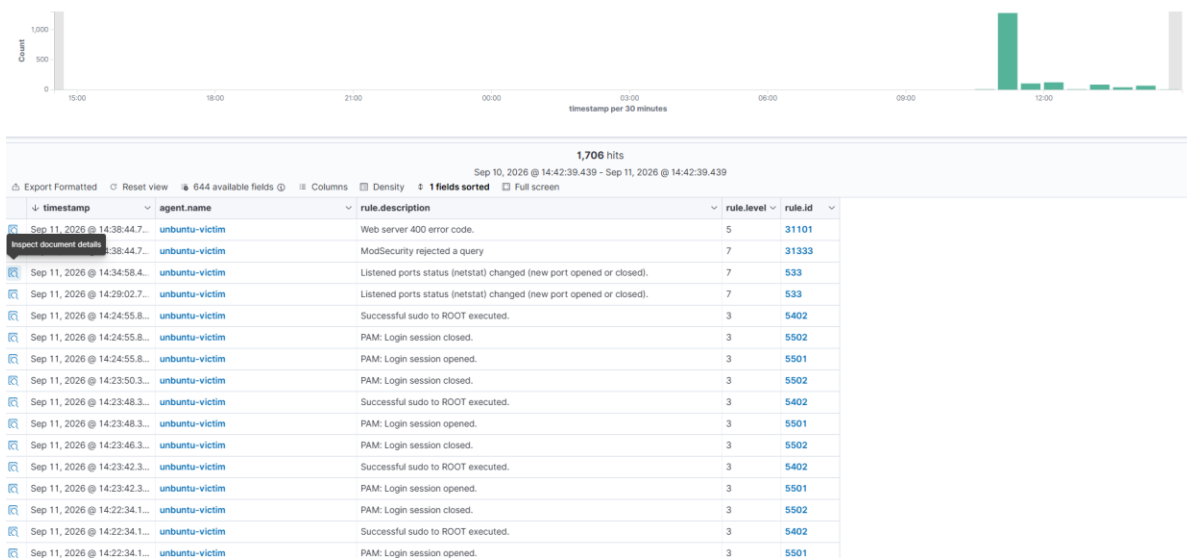
Payloads Tested

Payload	Technique	Result
127.0.0.1; whoami	Command chaining (semicolon)	Blocked (403)
127.0.0.1 whoami	Command chaining (pipe)	Blocked (403)
127.0.0.1 & cat /etc/passwd	Command chaining (ampersand)	Blocked (403)
...; echo <base64> base64 -d sh	Encoding evasion	Blocked (403)
127.0.0.1;cat\${IFS}/etc/passwd	Whitespace evasion (\$IFS)	Blocked (403)

I also submitted a legitimate, unmodified ping request (no injection) as a control, and it correctly passed through with a 200 response and normal output, confirming the WAF discriminates between malicious and legitimate traffic rather than blocking indiscriminately.

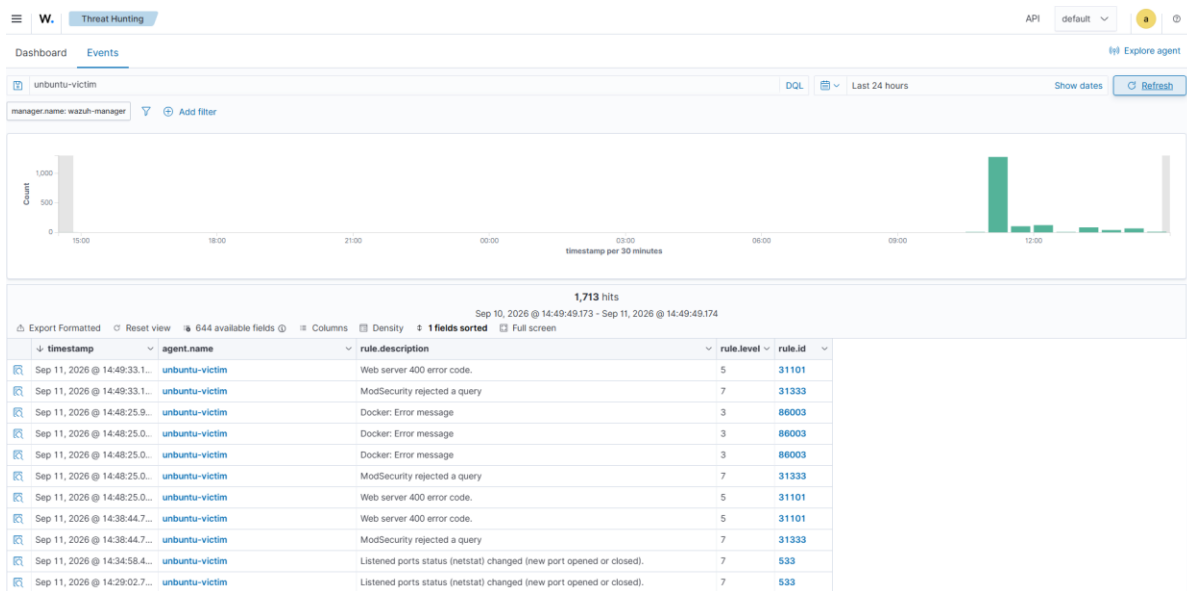
Detection Results in Wazuh

Every blocked attempt produced a corresponding alert: rule 31333 ("ModSecurity rejected a query", level 7), paired with rule 31101 ("Web server 400 error code") reflecting the resulting HTTP response.



Wazuh alerts confirming the WAF blocked each command injection attempt (rule 31333, paired with 31101).

A second cluster of test alerts also surfaced an unrelated informational event: rule 86003 ("Docker: Error message", level 3), traced to containerd's own internal logging (a benign ttrpc stream timing message), coincidental with the test window rather than caused by it.



Later test alerts, including the coincidental containerd informational log (rule 86003) investigated and ruled unrelated to the attack traffic.

Finding: A Rule Classification Gap

Every command injection attempt was blocked, the WAF held up consistently, including against both evasion techniques. However, unlike SQL injection, which triggers a dedicated classification rule (31103, "SQL injection attempt") in addition to the generic WAF block rule, Wazuh's default ruleset has no equivalent dedicated rule for command injection. Every blocked attempt here classified only as rule 31333, a generic "WAF rejected something" signal, with no rule level indication of which attack category triggered it.

This is a legitimate detection gap worth documenting: an analyst reviewing Wazuh alerts alone could see that ModSecurity blocked a request, but would need to inspect the raw `full_log` field manually to determine it was a command injection attempt specifically, rather than seeing that classification surfaced automatically the way SQL injection is.

Confirming Impact: What a Successful Injection Would Expose

Since all five payloads were blocked, the intended command output (e.g., `whoami`) was never returned through the injection itself. To confirm what that output would have been, I checked the web server's actual process owner directly on the host rather than through exploitation.

```

ubunvicti@unbuntu-victim:~$ ps aux | grep -i apache
root      1028  0.0  0.7 180824 19952 ?        Ss   19:01  0:02 /usr/sbin/apache2 -k start -DFOREGROUND
www-data  1482  0.0  0.3 181496 10272 ?        S    19:02  0:00 /usr/sbin/apache2 -k start -DFOREGROUND
www-data  1483  0.0  0.3 181496 10272 ?        S    19:02  0:00 /usr/sbin/apache2 -k start -DFOREGROUND
www-data  1492  0.0  0.3 181496 10272 ?        S    19:02  0:00 /usr/sbin/apache2 -k start -DFOREGROUND
www-data  1493  0.0  0.3 181496 10272 ?        S    19:02  0:00 /usr/sbin/apache2 -k start -DFOREGROUND
www-data  1494  0.0  0.3 181496 10272 ?        S    19:02  0:00 /usr/sbin/apache2 -k start -DFOREGROUND
root      3312  0.0  1.1 374000 30172 ?        Ss   19:06  0:01 /usr/sbin/apache2 -k start
www-data  3412  0.0  0.5 374456 14780 ?        S    19:06  0:00 /usr/sbin/apache2 -k start
www-data  3413  0.0  0.5 374612 15356 ?        S    19:06  0:00 /usr/sbin/apache2 -k start
www-data  3415  0.0  0.5 374440 14544 ?        S    19:06  0:00 /usr/sbin/apache2 -k start
www-data  3416  0.0  0.5 374476 14244 ?        S    19:06  0:00 /usr/sbin/apache2 -k start
www-data  3417  0.0  0.5 374620 15340 ?        S    19:06  0:00 /usr/sbin/apache2 -k start
root      3424  0.0  0.0  4248  1536 ?        S    19:06  0:00 tail -f /var/log/apache2/access.log /var/log/apache2/
error.log /var/log/apache2/other_vhosts_access.log
www-data  5524  0.0  0.5 374620 14860 ?        S    19:30  0:00 /usr/sbin/apache2 -k start
ubunvic+  6644 21.8  0.0  6716  2708 pts/0    S+   20:08  0:00 grep --color=auto -i apache
ubunvicti@unbuntu-victim:~$

```

Apache's worker processes confirmed running as `www-data`, the account any successful command injection would have executed under.

This confirms the realistic impact ceiling of this vulnerability class on this host: a successful injection would return output as `www-data`, a low privilege service account, rather than `root`. In a real assessment this would be the pivot point into further enumeration (checking `sudo -l`, group memberships, or writable `root` owned paths) rather than immediate full compromise,

useful context for scoping the actual severity of this class of finding rather than treating "command execution" as automatically total compromise.

MITRE ATT&CK Mapping

Framework	Mapping
MITRE Technique	T1059, Command and Scripting Interpreter
MITRE Tactic	Execution
Related Technique	T1190, Exploit Public Facing Application (initial vector)

Results Summary

- Tested 5 command injection payloads (3 standard chaining operators, 2 signature evasion techniques) against DVWA's Command Injection module through the ModSecurity WAF
- All 5 payloads blocked (403); a legitimate control request correctly passed through, confirming accurate WAF discrimination
- Confirmed WAF resilience against basic evasion techniques (base64/pipe to shell and \$IFS whitespace substitution), not just literal known bad strings
- Identified a genuine Wazuh rule classification gap: command injection blocks surface only as generic WAF reject alerts (31333), unlike SQL injection's dedicated classification rule (31103)
- Independently confirmed the web server's execution context (www-data) via direct system inspection, establishing the realistic impact ceiling of this vulnerability on this host
- Investigated and correctly ruled out an unrelated coincidental log event (containerd/ttrpc) rather than assuming it was attack related

Skills Demonstrated

- OS command injection testing methodology, including standard operators and signature evasion techniques
- WAF behavior validation: distinguishing true coverage from simple pattern matching using deliberately evasive payloads
- SIEM rule coverage gap analysis across different attack classes hitting the same WAF
- Log triage discipline: investigating an unfamiliar alert before assuming relevance, and correctly ruling out a coincidental finding
- Impact scoping via direct system verification (process ownership) rather than assuming worst case severity from vulnerability class alone
- MITRE ATT&CK mapping for execution stage techniques